Image: craiyon.com

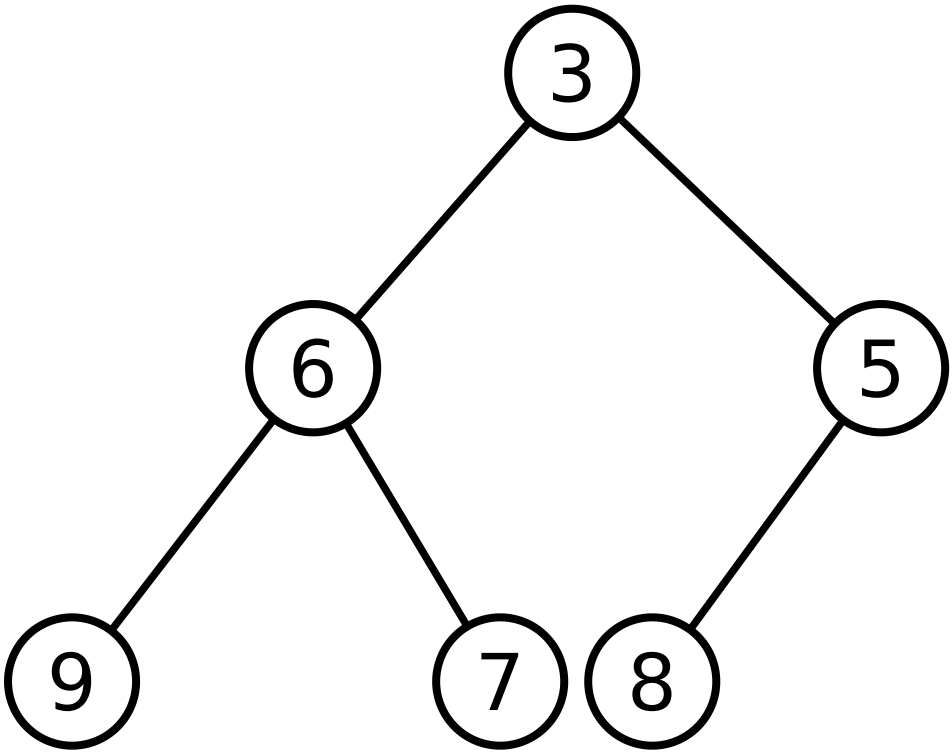
Priority queues and heaps

There is special type of queues called **Priority Queue**. But instead of giving back the elements that have been inserted/enqueue in order of insertion, the priority queue gives them by some criteria or priority that is defined for the queue. That could be, for example, to give the lowest value first or some other criteria that we could think off and can be applied to the data.

Even if they are queues in some sense, Priority Queues are usually implemented with **Heaps**. Heap is a Graph or Tree -like structure that satisfies the heap property. Heap property says that given a certain relationship that must exist between parent node/vertex and children, the parent should be less or equal than children. The consequence of this is that heaps have always the lowest value at the Root node. There is also the concept of a Max Heap, which is the inverse: the maximum value is at the top of the tree (normally heaps are actually Min Heaps).

Another characteristic of heaps is that parents should have at most two children. So, this means that heaps are Binary Trees. And one last property of heaps, is that heaps are **complete**. That means that they are filled from top to bottom and left to right. To start a new level the previous level should be full. Insertions and deletions should also take this into account and move nodes if necessary, so the resultant structure still works as a heap. One interesting thing is that instead of using nodes as we have seen with Trees, heaps are usually implemented using arrays.

Ok. Let's see a heap:

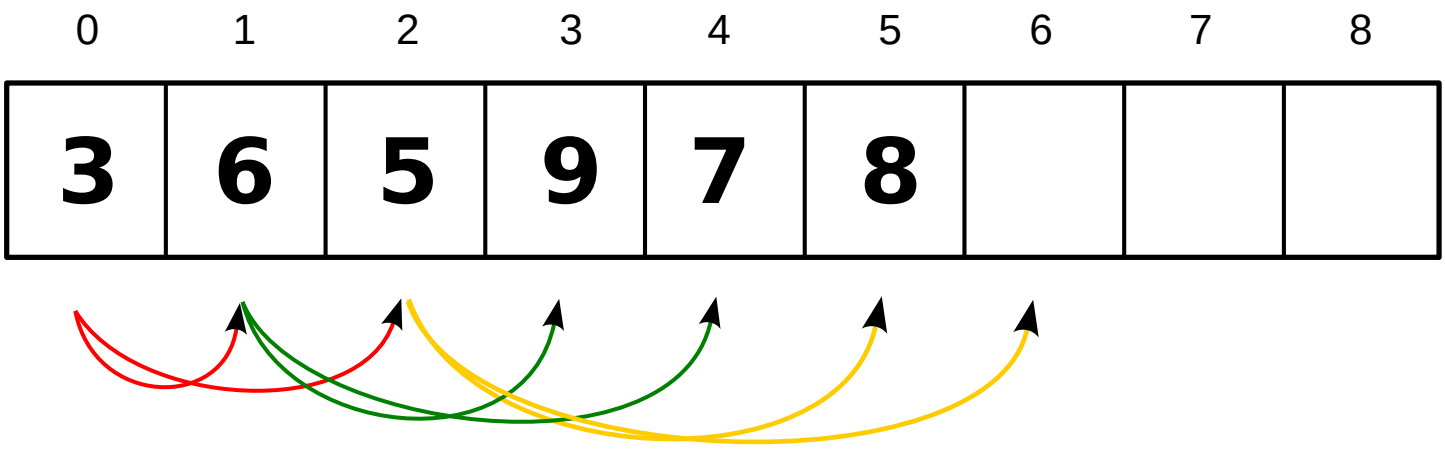


In the image we see that the minimum value is at the Root node. The rules doesn't say anything about the maximum value or the order, other than the parents should be of lesser value than their children: 3 is less than 6 and 5. 6 is less than 9 and 7. And 5 is less than 8. The heap is also filled from left to right at each level (and levels should be full before continuing with the next). So, this seems to be a valid heap.

How could this heap be stored in memory? As we said before, we use an array for that. The elements are stored in order from level 0 forward and from left to right:

0	1	2	3	4	5	6	7	8
3	**6**	**5**	**9**	**7**	**8**			

The advantage of this method is that it is possible to find the position of a node given its parent's position. If parent's position in the array is x, then its children are in the positions $2x + 1$ and $2x + 2$. So for position 0 ($x = 0$), children are 1 and 2. For position 1 ($x = 1$), children are 3 and 4. For position 2, children are 5 and 6. And so on...



This arrange gives us a good look up time. Given a node, it's trivial to get its children. It's time complexity $O(1)$.

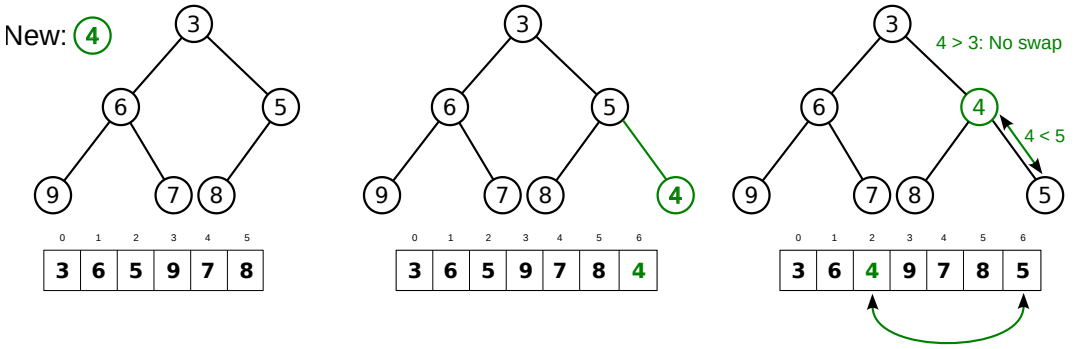
The implementation in Python is quite straightforward:

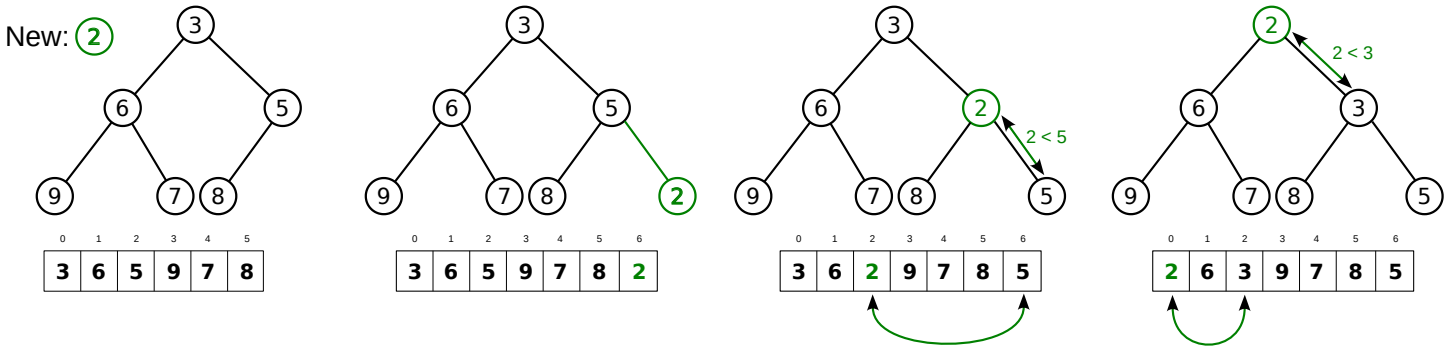
```
class Heap:
    def __init__(self):
        self._heap = []
        self._size = 0
```

Inserting and float

To insert a new value, it's easy. Create a new node and put it in the next free position. But we have to make sure, that the heap properties are still fulfilled. The main concern is that the new value could be less than its parent. If that happens, a process called floating comes into action, the new value it's **floated** until the heap properties are fulfilled. This operation is called heap-up, sift-up, bubble-up or some other terms.

Floating process it's very straightforward, just compare the node with its parent and swap the values if the child node is bigger than the parent. And repeat until the new node is less than its parent or it is the Root node.





This a perfect candidate for you to implement! The only important information to know is how to find the parent index in the array given a child. It's easy, but if you haven't realized yet, it's `(childnode_index-1) // 2`

Exercise!

Given the information you have received about floating, implement a float method that takes the last element of the heap and "float" it until the structure follow heap requirements.
Given a heap like: `[3, 6, 5, 9, 7, 8, 2]`, the new heap would be: `[2, 6, 3, 9, 7, 8, 5]`

Don't continue until you have finished the exercise!

A possible implementation is:

```
    def _float(self):
        index = self._size - 1
        while index > 0:
            parent_index = (index-1) // 2
            if self.heap[index] < self.heap[parent_index]:
                self.heap[index], self.heap[parent_index] = self.heap[parent_index],
self.heap[index]   # Swap
                index = parent_index
```

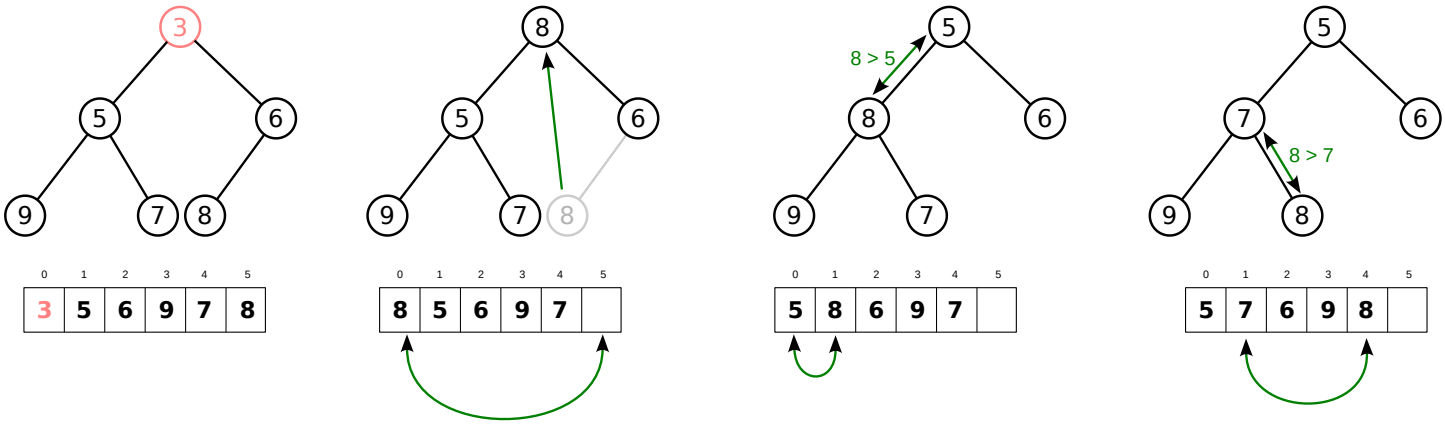
Let's continue with the insertion, but there is not much left. The insertion is then the process of adding the new element at the end of the heap and floating it if necessary. The implementation is almost literally that:

```
    def insert(self, value):
        self.heap.append(value)
        self._float()
        self._size += 1
```

The `self._float()` method is called anyway, because it will check if element needs floating and nothing will be done if not.

Popping and sinking

As said in the beginning heaps are a priority queue, that is, a queue after all. So the other action that we must implement besides the insertion, is the extraction of the first element on the queue. This was called a **Pop**. That element we want to extract is the Root node, but once we extract that element, what to do? How do we get back a working heap? Well, it's very easy. We take the last element and put in the Root node. But now we have to do the opposite we did previously, now we have to **sink** this value until the heap meets again the requirements. The sinking process is also called heap-down, sift-down, bubble-down or some other terms.



The process is not as straightforward as before, but it's very easy, nonetheless. Starting from the root node we have to compare the value of the node with the lesser value of the children. And as done with the float, swap values if necessary to maintain the heap integrity. Maybe the most delicate part of this process is finding the child with lesser value, because a node can have two children, one child or no child at all. Anyway, being careful with that, everything else it's easy.

Exercise!

Given the information you have received about extraction and sink, implement a `_sink` method that sinks the Root node value if necessary.
Given a heap like: `[8, 6, 5, 9, 7]`, the new heap would be: `[5, 6, 8, 9, 7]`

Don't continue until you have finished the exercise!

```
    def _sink(self):
        index = 0
        # While node has at least one child
        while index*2+1 < self._size:
            if index*2+2 < self._size:
                # If two children
                child_index = min(index*2+1, index*2+2, key=lambda x: heap[x])
            else:
                # If only one child
                child_index = index*2+1
            # Swap values if parent is bigger than child
            if heap[index] > heap[child_index]:
                self.heap[index], self.heap[child_index] = self.heap[child_index],
self.heap[index]   # Swap
                index = child_index
```

After that, the `pop()` method it's very easy:

```
    def pop(self):
        min_value = self._heap[0]
        self._heap[0] = self._heap.pop()
        self._size -= 1
        self._sink()
        return min_value
```

Analysis of a heap

The heap exposes publicly two methods `pop` and `insert`. Both methods work in a similar way.

The `insert` method adds a new element at the end of the heap. If the heap is implemented with a fixed array, time complexity is $O(1)$ (but array is limited in size). If implemented with a dynamic array, it is $O(1)$ of expected time, meaning that some of the insertions could take $O(n)$ time to expand the array. And if the heap is implemented with a Doubly linked list, the time complexity would be $O(1)$. The floating process at worst will move the last element from the end to the Root node. So this depends on the height of the tree. As seen before, height of the tree (given that it is complete) is $\log n$. Given that the access time for an element in the tree is $O(1)$, time complexity for floating is $O(\log n)$

Something similar happens with the `pop` method. It access the first element to retrieve it ($O(1)$) and then remove the last element to move it the Root node. That action would be of time complexity $O(1)$ or $O(1)$ expected time, depending on how is implemented the heap. The sinking of the element is

similar to the floating, and it depends on the height of the heap. As we have seen, that makes this operation of time complexity $O(\log n)$.

Therefore, for both methods the global time complexity is $O(\log n)$.

Searching

One area that needs to be addressed is searching. In many occasions we need to find a value in our dataset. There are a few different methods to perform a search, but as always, the most suitable method for a problem depends on the characteristics of the problem.

Let's see some of the methods!

Linear search

Linear search is the most basic and simple method of finding a value in a list/array and it is the only option if the list is not sorted. We just traverse the array searching for the method. If the value if found, return the position or index of the element. There is also a variant for when the list is sorted. In an unsorted list, there is no other option that iterate until the end to be sure if the value is there or not. So, it's easy to infer that the time complexity for each possible case (best case, worst case, average case) is $O(n)$ as the whole array should be iterated always if the value is not found.

```
def linear_unsorted_search(array, value):
    # Iterate array
    for index in range(len(array)):
        # If value is found, return its index
        if array[index] == value:
            return index
    return None
```

In a sorted array, the search can be interrupted if the element being checked is bigger than the value being searched, because that means that the value is not in the array (as the array is sorted).

```
def linear_sorted_search(array, value):
    # Iterate array
    for index in range(len(array)):
        # If value if found, return its index
        if array[index] == value:
            return index
        # If current array value is bigger than value, return None
        if array[index] > value:
            return None
    return None
```

So for the worst case (value being searched bigger than the values in the array) is still $O(n)$, but average case can be $O(\frac{n}{2})$, that is technically $\theta(n)$, but we can see that is better than the worst

case. The best case is that the value is found immediately (or first array value is bigger than the searched value). In that case time complexity is $\Omega(1)$.

Binary search

When an array/list is sorted there are more possible methods. Binary search tries to minimize the area being searched by splitting the array in two halves. A middle point is selected. If the middle point is not the searched value, then make the search in one of the halves, depending on whether the searched value is bigger or smaller than the middle point value. And repeat the process until the value is found or there is no half to be searched.

This algorithm can be implemented recursively or iteratively.

```
def binary_search_recursive(array, value, start=None, end=None):
    # Prepare the variables this way, so the user can call
    # the function with just the array and value to be searched.
    # If start and none are not defined, the search is conducted
    # in the whole array.
    if start is None:
        start = 0
    if end is None:
        end = len(array) - 1

    # Calculate the midpoint
    midpoint = start + (end - start + 1) // 2

    # If the value if found in the midpoint, return its index
    if array[midpoint] == value:
        return midpoint

    # if the value being searched is smaller than the one in the midpoint
    # and there is at least one element left to the midpoint
    if value < array[midpoint] and midpoint >= start+1:
        # Perform the search on the left half
        return binary_search_recursive(array, value, start=start, end=midpoint-1)

    # if the value being searched is bigger than the one in the midpoint
    # and there is at least one element right to the midpoint
    if value > array[midpoint] and midpoint <= end-1:
        # Perform the search on the right half
        return binary_search_recursive(array, value, start=midpoint+1, end=end)

    return None
```

The iterative version can be even easier but still is quite similar.

Exercise!

Try to implement an iterative version of the binary search. Use two pointers/variables to limit the search area and update them adequately. Loop while the search area is big enough to hold at least one element inside it.

Don't continue until you have finished the exercise!

A possible solution is:

```
def binary_search_iterative(array, value):
    # Search the whole array. Prepare variables.
    start = 0
    end = len(array) - 1

    # While start var is still left of end var, enter the loop
    while start <= end:
        # Calculate the midpoint
        midpoint = start + (end - start + 1) // 2

        # If the value if found in the midpoint, return its index
        if array[midpoint] == value:
            return midpoint

        # If the value being searched is smaller than the one in the midpoint
        if value < array[midpoint]:
            # move the end to the left, just before the midpoint
            end = midpoint - 1

        # If the value being searched is bigger than the one in the midpoint
        if value > array[midpoint]:
            # move the start to the right, just after the midpoint
            start = midpoint + 1

    # Value has not been found
    return None
```

The maximum number of searches with any of both implementations is $\log_2 n$ as in any level the number of positions to be search is halved in two. This is the same we have seen with the height of binary trees. The base 2 of the logarithm is usually taken implicitly, so the time complexity can be represented as $O(\log n)$.

Interpolation

Interpolation is a variant of the Binary search. In the Binary search the midpoint of the array is selected without considering the values on the array. Interpolation tries to fix that by calculating a "midpoint" not based on the position but on the values in the array.

For example, given an array like this:

```
    array = [10, 17, 25, 50, 90, 140, 200]
```

And if we want to search for the value 140, the Binary search would have selected as midpoint the value 50 at position 3. But given that the final value is 200, we can estimate that it should be closer to the end.

So, a better approximation could be:

```
midpoint = start + int((end - start) * ((value-array[start])/(array[end]-array[start])))
```

Being `start` the index of the first element, `end` the index of the last element, `array` the array in question and `value` the value being searched. Applying this formula, we get the index position 4. That it is not the right value, but it is closer to the correct one.

Even if the implementation is almost the same than in the Binary search, try to implement your version of the Interpolation search. Take into account that the calculated midpoint is not always within the limits of the search area.

Exercise!

Applying what we have just seen, implement an Interpolation search function.

Don't continue until you have finished the exercise!

```python
def interpolation_search(array, value, start=None, end=None):
    # Prepare the variables this way, so the user can call
    # the function with just the array and value to be searched.
    # If start and none are not defined, the search is conducted
    # in the whole array.
    if start is None:
        start = 0
    if end is None:
        end = len(array) - 1

    # Calculate a midpoint
    midpoint = start + int((end - start) * ((value-array[start])/(array[end]-array[start])))

    # If the calculated midpoint is not within the search area, return
    if midpoint < start or midpoint > end:
        return None

    # If the value is found at the midpoint, return the index of midpoint
    if array[midpoint] == value:
        return midpoint

    # if the value being searched is smaller than the one in the midpoint
    # and there is at least one element left to the midpoint
    if value < array[midpoint] and midpoint >= start+1:
        # Perform the search on the left part
        return interpolation_search(array, value, start=start, end=midpoint-1)

    # if the value being searched is bigger than the one in the midpoint
    # and there is at least one element right to the midpoint
    if value > array[midpoint] and midpoint <= end-1:
        # Perform the search on the right part
        return interpolation_search(array, value, start=midpoint+1, end=end)

    return None
```

The analysis of this algorithm reveals, that the best case is the same as in the Binary search, with time complexity $\Omega(1)$. The worst case is worst, because a very skewed dataset can drive this algorithm to a linear search, giving a time complexity of $O(n)$ (compared with the $O(n \cdot \log n)$ of the Binary search). But the average case is better, with a time complexity of $\theta(\log(\log n))$ (We will skip the mathematical demonstration in this case).